Tarea 1: Algoritmos de Ordenamiento Secuenciales y Paralelos

Javier Torres & Juan Umaña & Benjamin Jimenez
Sistemas Distribuidos y Paralelos
Universidad de Concepción

Abril 2026

Resumen

Este informe presenta la implementación, análisis y comparación de algoritmos de ordenamiento Mergesort y K-way Mergesort en sus variantes secuenciales y paralelas. Se utiliza el estándar OpenMP para la paralelización y se emplea la técnica de Merge por Ranks para optimizar la fusión de subarreglos. Los resultados experimentales, obtenidos sobre un procesador AMD Ryzen 5 8600G, validan el análisis teórico de Work y Span, demostrando una escalabilidad significativa en escenarios con alta carga de datos.

1. Introducción

El problema del ordenamiento es fundamental en la computación paralela, sirviendo como caso de estudio para el balance de carga, la granularidad y la gestión de memoria. En este trabajo, se exploran estrategias para minimizar el *Span* de los algoritmos clásicos, pasando de una recursión paralela simple a una implementación completa que paraleliza incluso la operación de mezcla (*merge*).

2. Descripción de Algoritmos

2.1. Mergesort Secuencial y Paralelo

El Mergesort se basa en el paradigma de divide-and-conquer. La versión paralela implementada utiliza tareas (*tasks*) de OpenMP para procesar las llamadas recursivas de forma concurrente. Se introduce un parámetro de **granularidad** para evitar el sobrecosto de creación de hilos en subproblemas pequeños.

2.2. Merge Paralelo por Ranks

Para superar el cuello de botella del merge secuencial de $O(n)$, se implementó el algoritmo de **Merge Path**. Este utiliza búsqueda binaria para calcular los *ranks* (puntos de corte) en los subarreglos ordenados, permitiendo que múltiples hilos fusionen tramos independientes de forma simultánea.

2.3. K-way Mergesort

A diferencia del Mergesort binario, el K-way divide el arreglo en k bloques. La versión paralela completa combina la división en k partes con el merge paralelo interno, maximizando el aprovechamiento de los núcleos disponibles.

3. Análisis Teórico

3.1. Modelo PRAM y Work/Span

Considerando un arreglo de tamaño n y p procesadores:

- **Mergesort Paralelo (Merge Secuencial):**
  - $W(n) = O(n \log n)$
  - $S(n) = S(n/2) + O(n) = O(n)$
- **Merge Paralelo por Ranks:**
  - $W_{merge}(n) = O(n)$
  - $S_{merge}(n) = O(\log n)$
- **Versión Paralela Completa:**
  - $S(n) = S(n/k) + O(\log n) = O(\log^2 n)$ (para k constante).

3.2. Teorema de Brent

Según el Teorema de Brent, el tiempo de ejecución estimado es $T_p \leq \frac{W(n)}{p} + S(n)$. En nuestra implementación completa, esto se traduce en $T_p \approx \frac{n \log n}{p} + \log^2 n$, lo que predice una alta eficiencia para valores grandes de n.

4. Metodología Experimental

4.1. Entorno de Ejecución

Las pruebas se realizaron en un procesador **AMD Ryzen 5 8600G** (6 núcleos físicos, 12 hilos) con 32 GB de RAM DDR5. Se utilizó el compilador ‘g++‘ con el flag de optimización ‘-O3‘.

4.2. Parámetros de Prueba

- Tamaños de arreglo: $n \in \{2^{20}, 2^{22}, 2^{24}, 2^{26}\}$.
- Hilos: $p \in \{1, 2, 4, 8\}$.
- Granularidad: 8192 elementos.

5. Resultados y Discusión

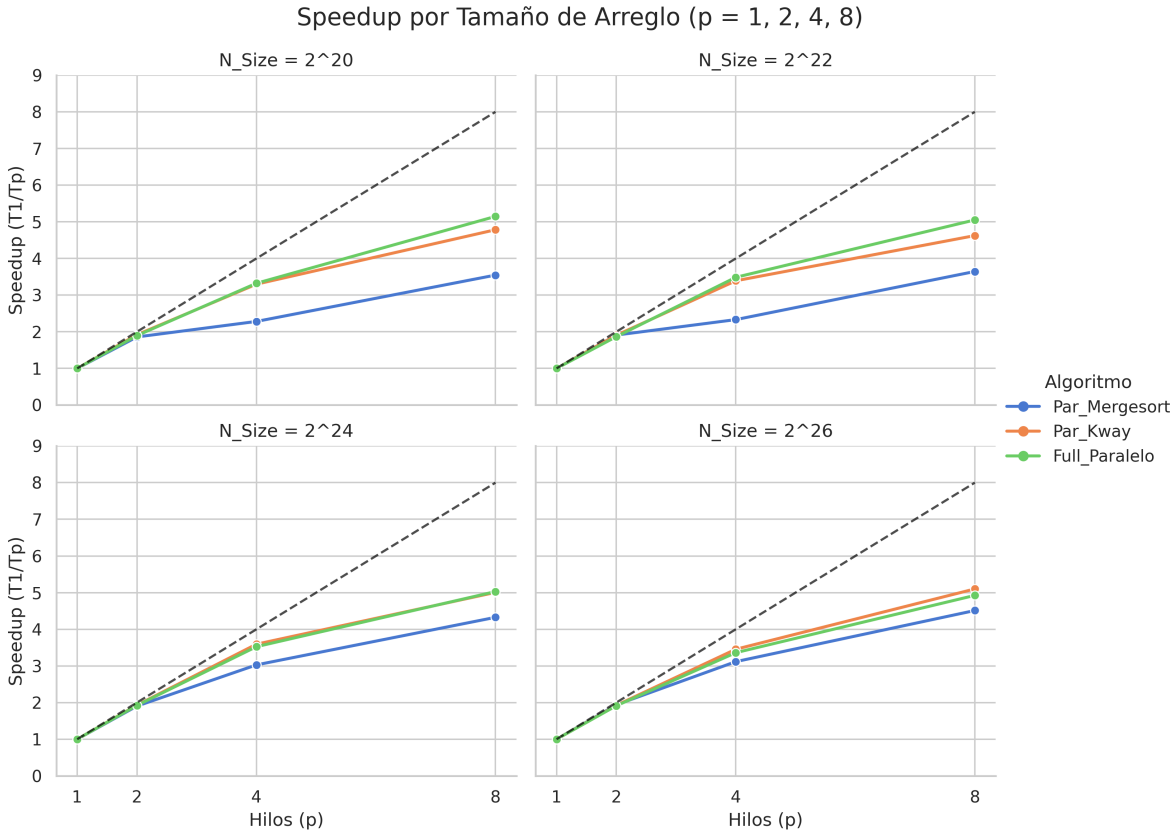

Figura 1: Speedup obtenido para diferentes tamaños de arreglo y número de hilos.

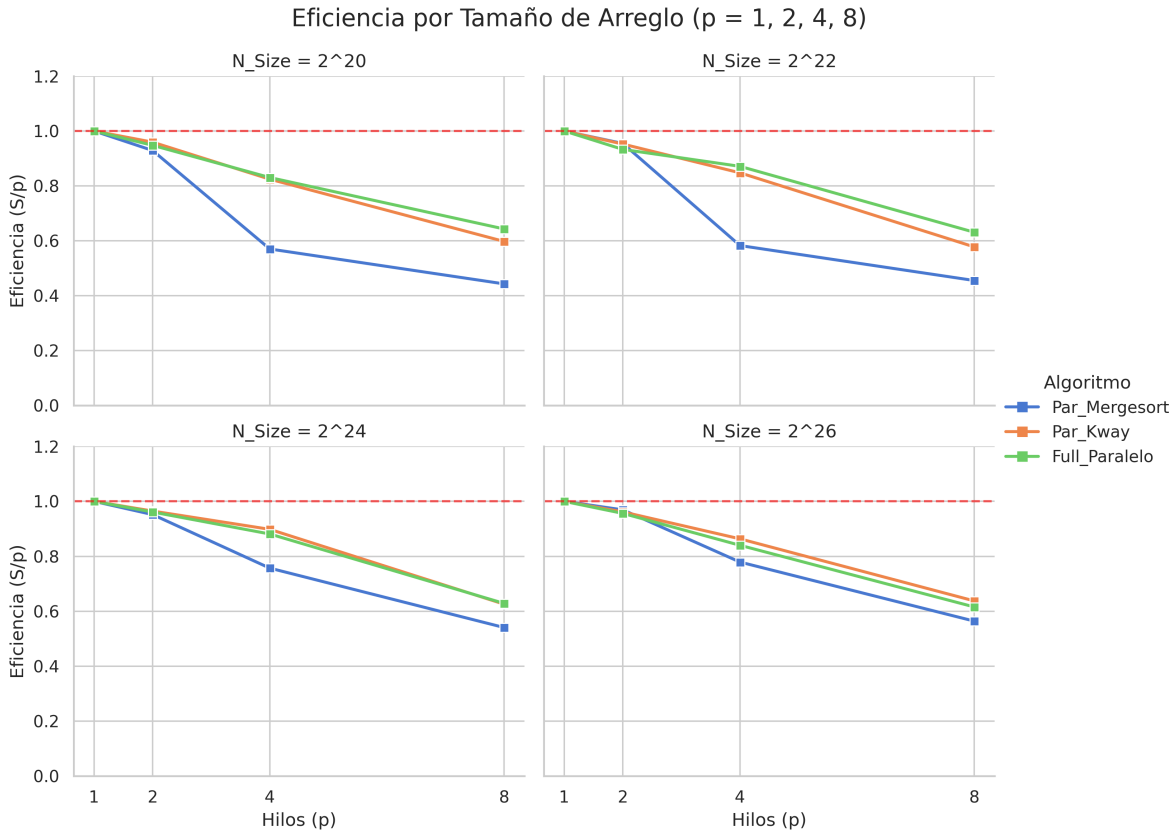

Figura 2: Eficiencia relativa en función del número de hilos y tamaño del arreglo.

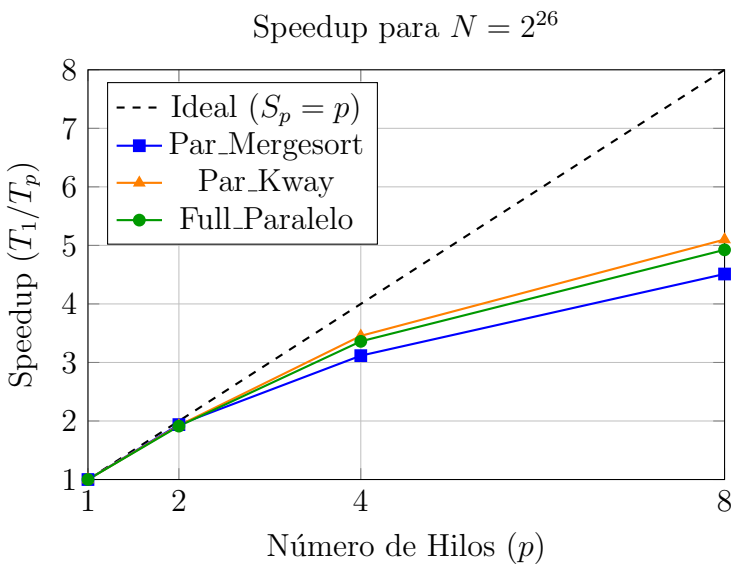

Figura 3: Escalabilidad de los algoritmos para el tamaño máximo de arreglo analizado. Se observa un estancamiento de la eficiencia al utilizar 8 hilos, alejándose de la curva ideal.

6. Conclusiones

La implementación del merge paralelo es fundamental para lograr una escalabilidad real en arquitecturas multinúcleo. Los resultados demuestran que, aunque el *Work* se mantiene constante, la drástica reducción del *Span* permite que el sistema se acerque al speedup ideal en problemas de gran tamaño.